EMBEDDED SYSTEM DESIGN EXPERIMENT

# 嵌入式系统设计实验

曾毓

## 实验 08 STM32定时器应用实验

杭州電子科技大学
HANGZHOU DIANZI UNIVERSITY

# 定时器基本概念介绍

定时器可以看作对周期脉冲信号的计数单元，在单片机中定时器通常是对外设时钟信号进行计数。

使用定时器时通常需要注意3点：**位宽**、**计数值**和**计数溢出操作**。计数溢出操作是定时器计数溢出产生溢出信号时需要完成的操作，对单片机而言即是定时中断服务函数中的执行动作。

传递给定时器的周期性时钟信号，其频率通常称为定时器的时钟频率，定时器计数单元计一次数花费的时间即是该时钟频率的倒数。

定时器的定时周期通常是指定时器的定时中断间隔时间，**定时周期的计算**方法一般如下：**（计数终值-计数初值+1）/时钟频率**。如168 MHz的时钟频率下，计数初值为0、终值为167时，定时周期=(167-0+1)/168 MHz=1 μs。

# STM32定时器基本概念介绍

常规定时器的时钟源选择包括四种：**内部时钟、外部输入引脚、外部触发输入ETR和内部触发信号**。内部时钟是定时器时钟源的常见选择，其来自于外设总线APB提供的时钟。

STM32中，从时钟源连接到定时器这边的信号，经过定时器的预分频模块后产生的信号称为**计数时钟CK_CNT**，定时器内部的计数模块就是对CK_CNT进行计数。预分频模块的作用在于扩大定时器的定时范围和获取精确的计数时钟。例如TIM_CLK频率为168MHz时，可以通过预分频模块对TIM_CLK进行分频得到1MHz的计数时钟。CK_CNT频率计算公式如下：

$$f_{CK_CNT} = f_{TIM_CLK} / (PSC + 1)$$

$f_{TIM_CLK}$为定时器时钟TIM_CLK的频率，**PSC**是预分频模块中的**预分频系数**，因此要得到1MHz的计数频率时，$f_{TIM_CLK}$得做168分频，PSC的值应该为168 - 1 = 167。

# STM32定时器基本概念介绍

定时器对CK_CNT信号的计数模式包括向上、向下和双向三种。在向上模式，每来一个CK_CNT的脉冲信号，计数值递增加1，当计数值达到**自动重装载寄存器ARR**的数值时产生计数溢出情况，此时计数值将进行清零操作。因此，定时器的定时时间计算公式为：

$$T = (ARR + 1) * (PSC + 1) / f_{TIM_CLK}$$

如要实现100毫秒的定时时间，在TIM_CLK为168MHz时，可以设置ARR和PSC两个参数值分别为1000 - 1和16800 - 1。

要注意，PSC取值不能超出**有效范围0 ~ 65535**，而ARR的位数由定时器位数决定，16位定时器的ARR数值范围也是0 ~ 65535，32位定时器的ARR数值范围0 ~ 4294967295。

# STM32F4定时器介绍

STM32F407有14个定时器，其中12个是16位定时器，2个是32位定时器。每个定时器都有通用定时计数、PWM输出、输出比较和输入捕获（TIM6、7没有）功能。

TIM1和TIM8两个高级定时器带有的互补输出功能常用于电机控制。

TIM2和TIM5两个32位定时器可用于测量高频信号频率。

| 名称 | 分类 | 位数 | 计数模式 | 捕获/比较通道 | 互补输出 | 默认时钟频率 |
|---|---|---|---|---|---|---|
| TIM1、TIM8 | 高级定时器 | 16 | 向上、向下，双向 | 4 | 有 | 168MHz |
| TIM2、TIM5 | 通用定时器 | 32 | 向上、向下，双向 | 4 | 无 | 84MHz |
| TIM3、TIM4 | 通用定时器 | 16 | 向上、向下，双向 | 4 | 无 | 84MHz |
| TIM6、TIM7 | 通用定时器 | 16 | 向上、向下，双向 | 0 | 无 | 84MHz |
| TIM9 | 通用定时器 | 16 | 向上 | 2 | 无 | 168MHz |
| TIM10、TIM11 | 通用定时器 | 16 | 向上 | 1 | 无 | 168MHz |
| TIM12 | 通用定时器 | 16 | 向上 | 2 | 无 | 84MHz |
| TIM13、TIM14 | 通用定时器 | 16 | 向上 | 1 | 无 | 84MHz |

# HAL库定时器模块设计应用方法

HAL库中定时器的设计应用方法，和之前章节的ADC、串口等复杂外设类似，主要包括外设句柄、编程模型和通用接口函数这三部分概念。

定时器外设句柄类型为TIM_HandleTypeDef，这个名称和ADC外设句柄类型相似，以外设名称开头，后面接_HandleTypeDef尾缀。HAL库函数操作外设时，需要对外设实例指针指向地址包含的外设寄存器进行读写操作，因此HAL外设相关函数的参数列表的第一个参数通常都是外设句柄指针。

定时器的外设编程模型按照HAL库的编程模型标准可分为轮询方式、中断方式和DMA方式。这三种编程模型相关函数如下表所示：

| 函数名称 | 说明 |
|---|---|
| **HAL_TIM_Base_Start**(TIM_HandleTypeDef *htim); | **启动定时器**，轮询方式 |
| HAL_TIM_Base_Stop(TIM_HandleTypeDef *htim); | 停止定时器，轮询方式 |
| **HAL_TIM_Base_Start_IT**(TIM_HandleTypeDef *htim); | **启动定时器并开启中断** |
| HAL_TIM_Base_Stop_IT(TIM_HandleTypeDef *htim); | 停止定时器及其中断 |
| HAL_TIM_Base_Start_DMA(TIM_HandleTypeDef *htim); | 启动定时器并开启DMA |
| HAL_TIM_Base_Stop_DMA(TIM_HandleTypeDef *htim); | 停止定时器及其DMA |

# STM32定时器的通用接口函数

定时器的通用接口函数，包括对定时器的参数配置、初始化、I/O操作和状态获取等函数，常用的定时器接口函数如下表所示。

| 函数名称 | 说明 |
| --- | --- |
| **HAL_TIM_PeriodElapsedCallback** | **定时溢出中断回调函数** |
| **__HAL_TIM_SET_COUNTER** | 设置定时器计数值 |
| **__HAL_TIM_GET_COUNTER** | 获取定时器计数值 |
| **__HAL_TIM_SET_AUTORELOAD** | 设置自动重装载寄存器数值 |
| __HAL_TIM_GET_AUTORELOAD | 获取自动重装载寄存器数值 |
| **__HAL_TIM_SET_PRESCALER** | 设置定时器预分频系数 |
| __HAL_TIM_SET_COMPARE | 设置定时器通道比较数值 |
| __HAL_TIM_GET_COMPARE | 获取定时器通道比较数值 |
| **HAL_TIM_PWM_Start**、HAL_TIM_PWM_Start_IT、HAL_TIM_PWM_Start_DMA | **启动定时器并开启PWM输出功能** |
| HAL_TIM_PWM_Stop、HAL_TIM_PWM_Stop_IT、HAL_TIM_PWM_Stop_DMA | 停止定时器PWM输出功能 |
| HAL_TIM_OC_Start、HAL_TIM_OC_Start_IT、HAL_TIM_OC_Start_DMA | 启动定时器输出比较功能 |
| HAL_TIM_OC_Stop、HAL_TIM_OC_Stop_IT、HAL_TIM_OC_Stop_DMA | 停止定时器输出比较功能 |
| HAL_TIM_IC_Start、HAL_TIM_IC_Start_IT、HAL_TIM_IC_Start_DMA | 启动定时器输入捕获功能 |
| HAL_TIM_IC_Stop、HAL_TIM_IC_Stop_IT、HAL_TIM_IC_Stop_DMA | 停止定时器输入捕获功能 |

# 定时中断回调函数

在CubeMX工程中添加FreeRTOS组件并**重新设置了HAL的系统时钟源**后，导出生成的main.c文件会在文件末尾自动生成HAL_TIM_PeriodElapsedCallback()函数，这是定时器的溢出中断回调函数，当定时器计数溢出发生中断时，在定时器的中断函数中会自动调用这个回调函数。生成的回调函数代码如下所示：

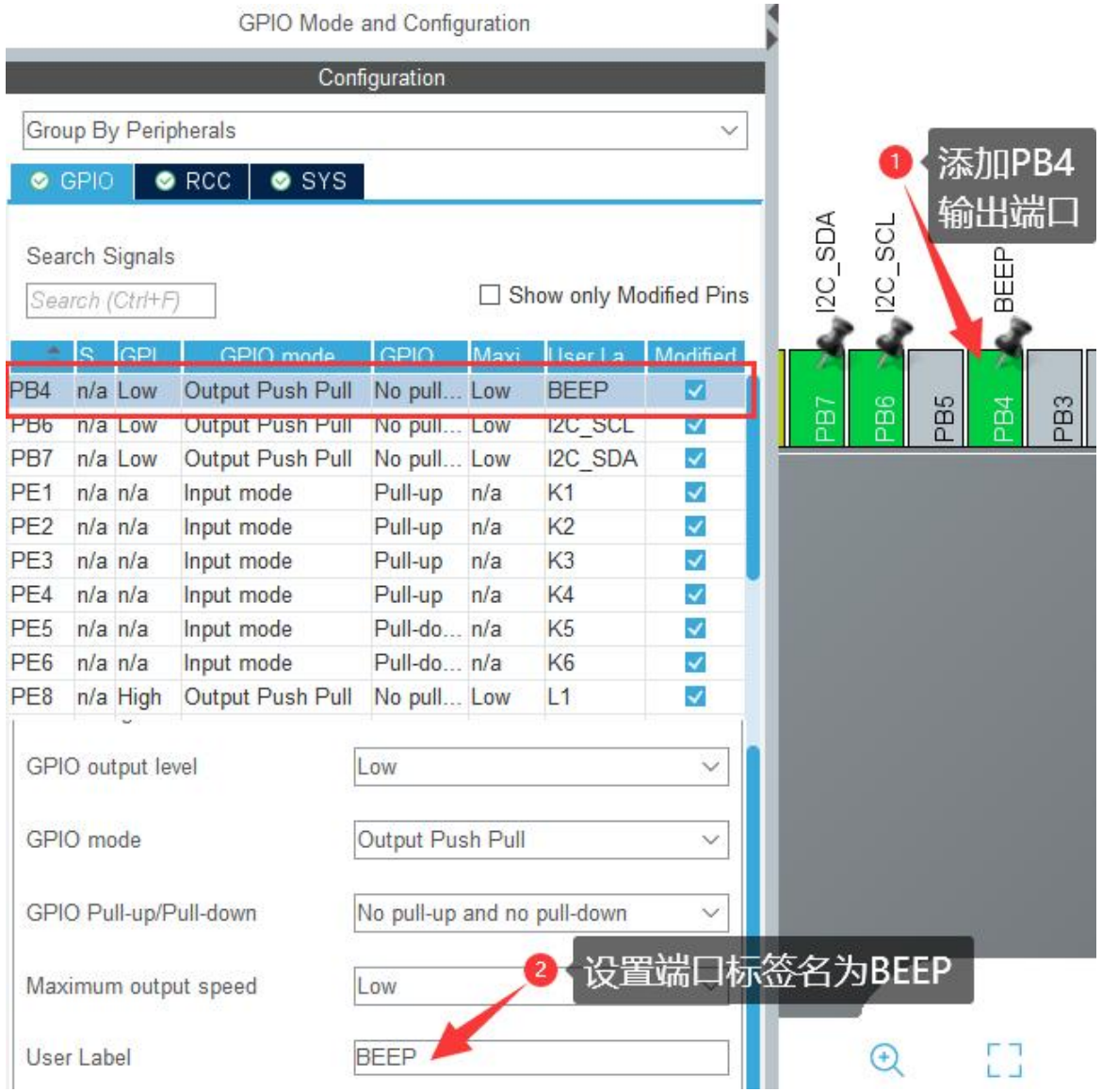
```
/* USER CODE END 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  /* USER CODE BEGIN Callback 0 */

  /* USER CODE END Callback 0 */
  if (htim->Instance == TIM7) {          // CubeMX中选择了TIM7作为HAL系统时钟源
    HAL_IncTick();
  }
  /* USER CODE BEGIN Callback 1 */
  /* USER CODE END Callback 1 */
}
```

# 简单电子琴设计示例——新建工程与添加端口

在实验7的EX07工程基础上，复制一份工程文件夹。并重命名工程为**EX08**，删除文件夹内的MDK-ARM子目录，将.ioc文件重命名为EX08.ioc。接下来准备实现一个简单的电子琴演示程序。

学习板上的蜂鸣器是一个无源蜂鸣器，需要给它一定频率的脉冲才能发出声音。根据这一特点，可以通过定时翻转BEEP端口的电平，输出连续脉冲信号给蜂鸣器就能鸣叫了。

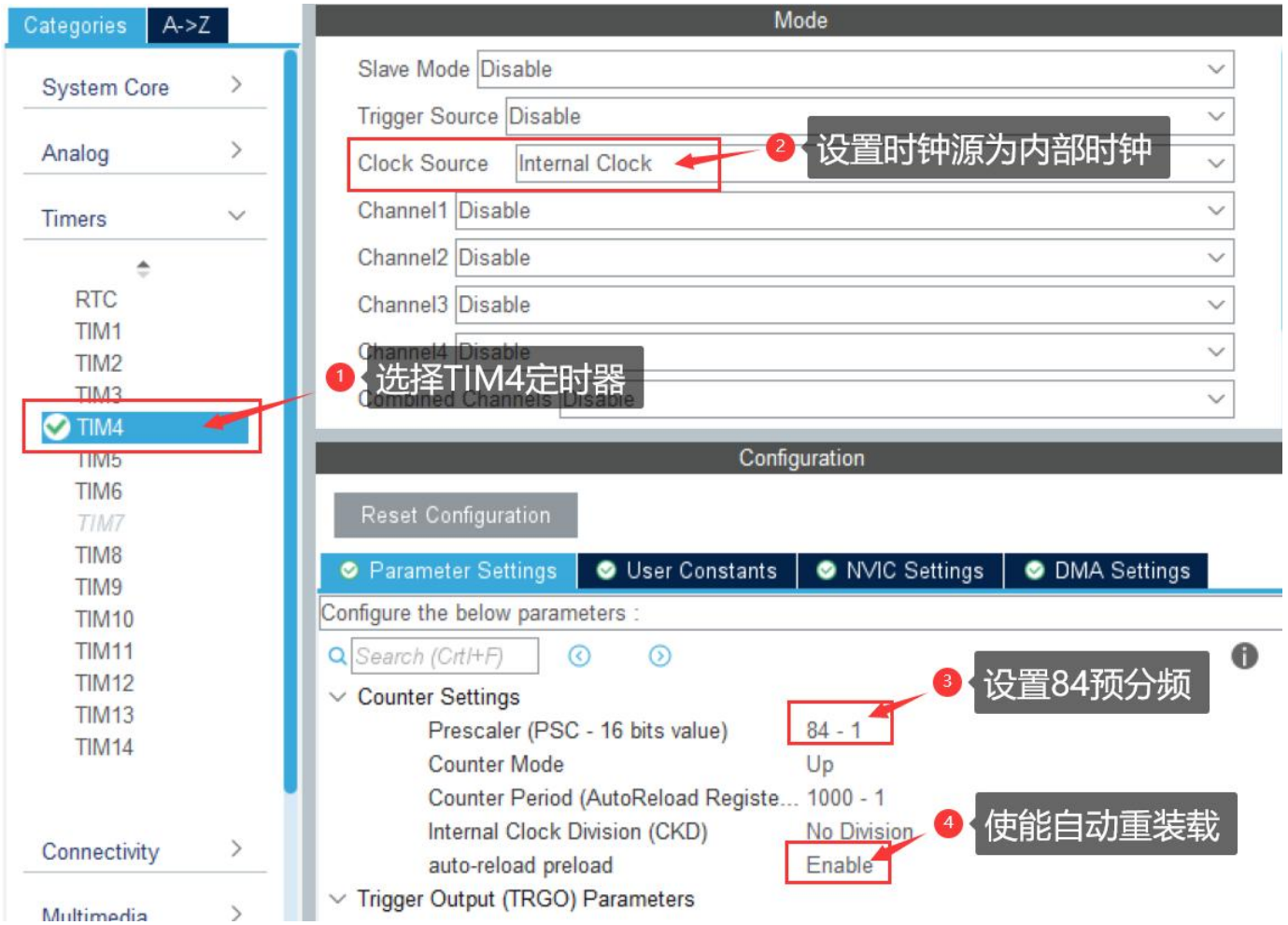
如右图所示，打开EX08的CubeMX工程，添加学习板上的蜂鸣器端口**PB4**，**输出端口**模式，命名为**BEEP**。



# 简单电子琴设计示例——定时器设置

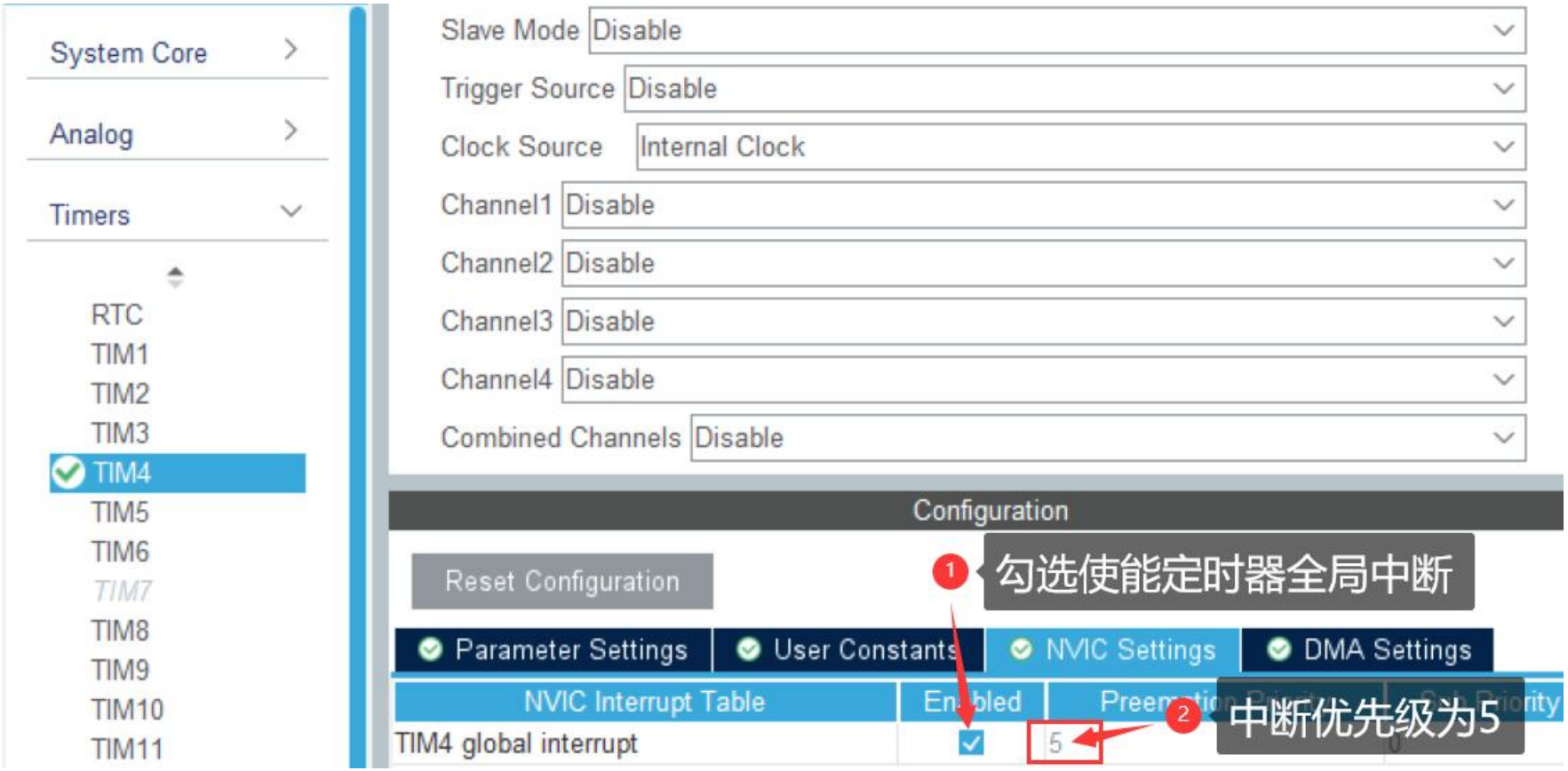
在左侧的定时器列表中，选择一个定时器用于控制蜂鸣器鸣叫，定时器参数设置如下图所示，注意TIM7因为用于SYS系统时钟基准源，所以是灰色不可选状态。

TIM4定时器的PSC参数设置为84-1，其用意是把TIM4的计数时钟频率降到1MHz，然后在程序中动态调整ARR寄存器就可以得到约8Hz ~ 250KHz的蜂鸣器脉冲信号了。



# 简单电子琴设计示例——使能定时器中断

TIM4定时器还需要开启定时中断，如下图所示，选择TIM4的NVIC设置页，勾选使能TIM4定时器的全局中断，注意其中断优先级应该默认为5。操作完成后，保持FreeRTOS模块的设置不变，导出EX08的MDK工程。



# 简单电子琴设计示例——添加蜂鸣器驱动代码

打开EX08的MDK工程，重新添加OLED驱动和图片、字体，设置工程包含路径。修改gpio.h和gpio.c文件，添加蜂鸣器鸣叫控制的全局变量和设置函数代码如下：

```
... // gpio.h文件添加变量和函数声明
/* USER CODE BEGIN Prototypes */
void SetLeds(uint8_t dat);
uint8_t ScanKey(void);
void Beep(uint8_t tune, uint16_t time);
extern uint8_t beep_tune;
extern uint16_t beep_time;
/* USER CODE END Prototypes */

... // gpio.c文件开头添加全局变量声明
/* USER CODE BEGIN 1 */
#include "tim.h"  // 添加定时器头文件
// 定义蜂鸣器鸣叫音调变量
uint8_t beep_tune = 0;
// 定义蜂鸣器鸣叫时长变量，单位毫秒
uint16_t beep_time = 0;
/* USER CODE END 1 */
...
```

```
... // gpio.c文件末尾添加Beep函数
void Beep(uint8_t tune, uint16_t time) { // 鸣叫函数
    // 音调对应频率表（C4～B4）
    const float tab[8] = {0, 261.6, 293.6, 329.6, 349.2, 392.0, 440.0, 493.9};
    tune %= 8;          // 限制音调范围0～7
    if (tune > 0) {     // 如果是有效音调
        HAL_TIM_Base_Start_IT(&htim4);
        // 根据目标频率计算ARR的值：(1M / tab[tune]) / 2 - 1
        float arr = (1000000 / tab[tune]) / 2 - 1;
        __HAL_TIM_SET_AUTORELOAD(&htim4, (uint16_t)arr);  // 重新调整定时频率
        beep_tune = tune;       // 保存音调
        beep_time = time;       // 设置鸣叫时长
    }
    else                    // 不是有效音调时，停止定时器
        HAL_TIM_Base_Stop_IT(&htim4);
}
/* USER CODE END 2 */
```

# 简单电子琴设计示例——修改定时中断回调函数

回到main.c文件，修改定时中断回调函数，添加BEEP端口的电平翻转控制代码：

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  /* USER CODE BEGIN Callback 0 */
  if (htim->Instance == TIM7) { // 如果是HAL库的基准时钟（1毫秒定时周期）
      if (beep_tune > 0 && beep_time > 0)        -- beep_time;     // 蜂鸣器时长控制，每毫秒递减
      else                                       beep_tune = 0;    // 时长为0时音调清零
  }
  /* USER CODE END Callback 0 */
  if (htim->Instance == TIM7) {
      HAL_IncTick();
  }
  /* USER CODE BEGIN Callback 1 */
  if (htim->Instance == TIM4) {  // 如果是TIM4定时器的中断
    if (beep_tune > 0)  HAL_GPIO_TogglePin(BEEP_GPIO_Port, BEEP_Pin); // 如果音调有效，翻转输出电平
  }
  /* USER CODE END Callback 1 */
}
```

# 简单电子琴设计示例——添加按键声音功能

最后，修改freertos.c文件中的按键任务，为每一个独立按键添加蜂鸣器鸣叫控制的逻辑代码。程序修改完后，重新编译，下载测试观察按键声音是否正常。

```
void StartKeyTask(void *argument) {
  /* USER CODE BEGIN StartKeyTask */
  osDelay(100);        Beep(7, 50);        // 任务开始鸣叫提示
  for(;;)  {
    uint8_t key = ScanKey();              // 扫描按键键码
    if (K1_Pin == key) Beep(1, 50);       // 6个按键对应不同的音调
    else if (K2_Pin == key) Beep(2, 50);
    else if (K3_Pin == key) Beep(3, 50);
    else if (K4_Pin == key) Beep(4, 50);
    else if (K5_Pin == key) Beep(5, 50);
    else if (K6_Pin == key) Beep(6, 50);
    if (key > 0) {
      osThreadFlagsSet(defaultTaskHandle, key);        // 键码发给默认任务处理
      ...                                              // 其余代码保持不变
```

# 定时器PWM输出功能介绍

PWM（脉宽调制）是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术，广泛应用于电机控制、灯光的亮度调节、功率控制等领域。脉宽调制的实质是修改高电平的持续时间，即调节占空比的大小来等效出所需要的波形。

STM32的高级定时器和通用定时器都具有PWM输出功能，每个定时器最多有四个通道，而每一个通道都有一个捕获比较寄存器，将寄存器值和计数器值比较，通过比较结果输出高低电平，从而实现输出PWM信号。因此，只要控制好捕获比较寄存器（CCRx）的数值大小，就可以得到不同占空比的PWM信号。

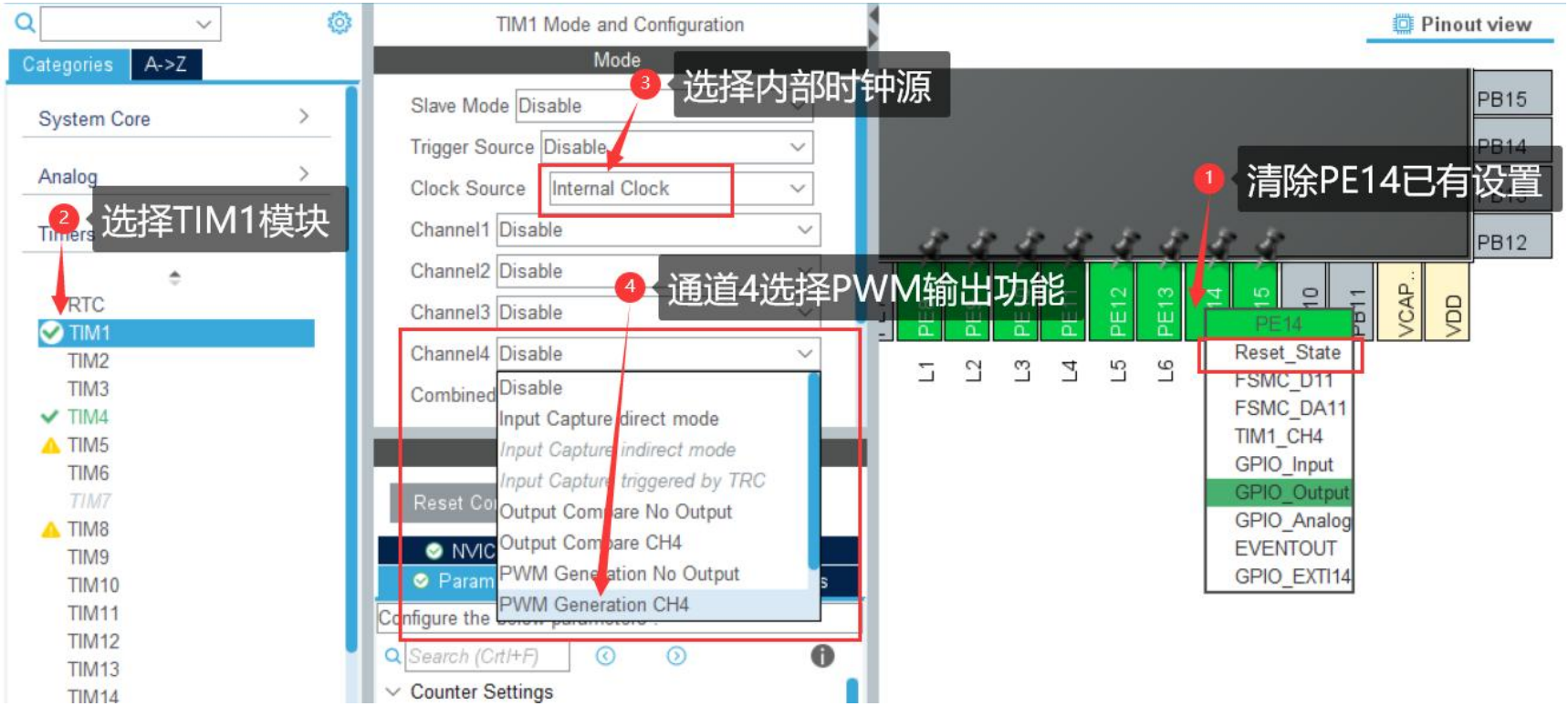
学习板上的4个LED灯（**L2、L4、L6、L7**）分别连接了STM32F407的TIM1定时器4个PWM输出通道。接下来将在EX07工程基础上，应用定时器的PWM输出功能，实现一个LED灯调光应用。

# LED灯调光应用示例——修改LED端口输出模式

配套学习板上的第7个LED灯L7，连接到了定时器1的通道4，本次示例选择L7作为调光演示的LED灯。

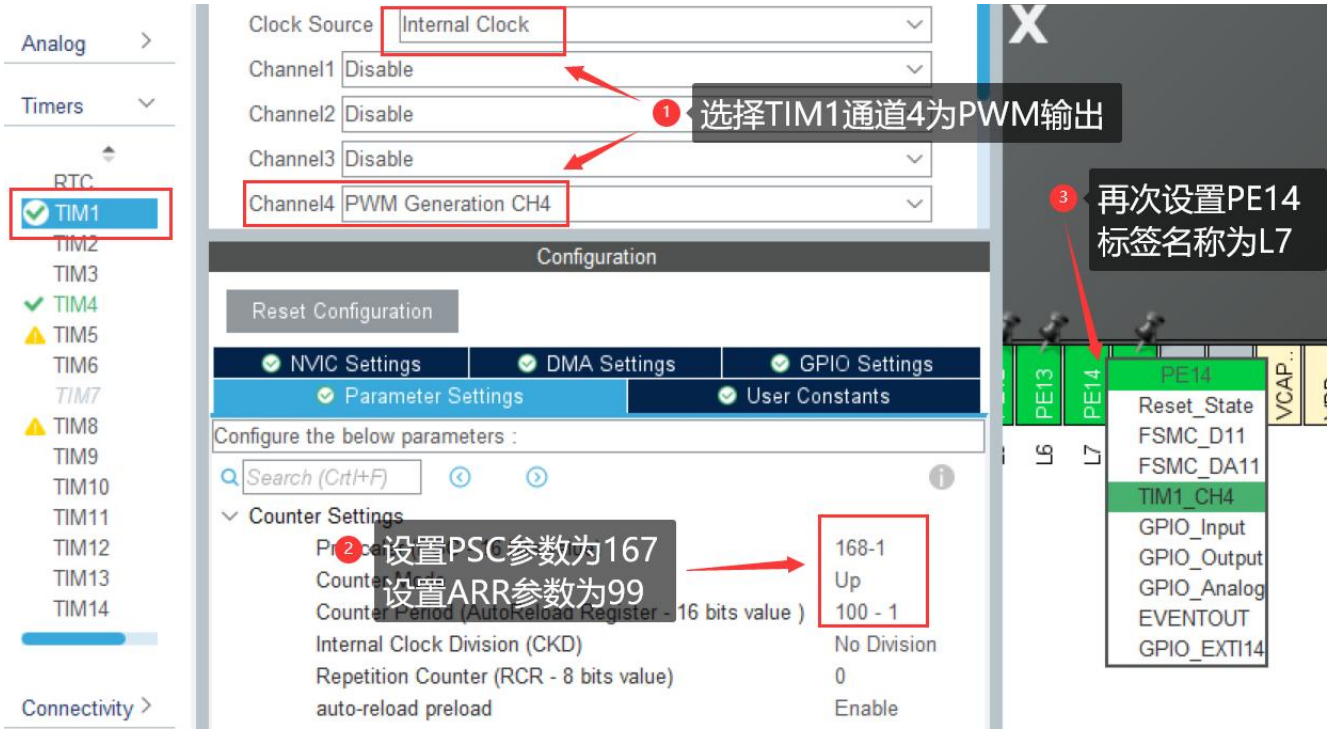
| LED名称 | 连接引脚 | 定时器功能 |
|---|---|---|
| L7 | PE14 | TIM1_CH4 |

重新打开EX08的CubeMX工程，如下图所示，在器件视图中，首先清除PE14引脚上的已有设置，然后使能TIM1模块，设置内部时钟源。



# LED灯调光应用示例——定时器1参数设置

PWM信号的两个基本参数是周期和占空比，周期是一个完整PWM波形所持续的时间，占空比（Duty）是高电平持续时间与周期时间的比值。对STM32单片机，PWM周期即是定时器的定时周期，之前已经提到可以通过设置ARR和PSC两个参数来调节定时周期。在CubeMX中，设置TIM1定时器参数如下图所示，设置**PSC和ARR为168-1和100-1**，将定时器周期设置为168x100/168MHz=0.1毫秒，即输出PWM信号的频率为10KHz。



# LED灯调光应用示例——包含头文件

设置定时器某个通道为PWM输出模式后，该通道有一个名为“Pulse”的PWM参数可调，输出PWM信号的占空比=(Pulse / ARR)×100%。上页中，如果Pulse值为50，输出PWM信号占空比即为50%，如果要实现动态调光效果，可以在程序中调用__HAL_TIM_SET_COMPARE()动态设置占空比。

打开freertos.c文件，先在文件开头添加包含tim.h头文件：

```
/* USER CODE BEGIN Includes */
#include "gpio.h"
#include "gui.h"
#include "tim.h"
/* USER CODE END Includes */
```

# LED灯调光应用示例——修改默认任务功能代码

接下来修改StartDefaultTask任务函数代码如右，设计实现K1、K4调节L7亮度，K2、K3的熄灯、亮灯功能。

因为LED灯是低电平有效，所以按键K1减小占空比使得灯L7越来越亮，反之按键K4加大占空比使得灯L7越来越暗。

```
void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, 80); // 设置初始占空比80%
  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_4);          // 启动定时器1的PWM输出功能
  /* Infinite loop */
  for(;;) {
    int key = osThreadFlagsWait(KEY_MASK, osFlagsWaitAny, 0);     // 等待按键消息
    if (key > 0 && key <= KEY_MASK) {
      uint16_t pulse = __HAL_TIM_GET_COMPARE(&htim1, TIM_CHANNEL_4); // 读占空比
      switch (key) {
        case K1_Pin: if (pulse > 0) pulse -= 2; // L7亮度增加（低电平亮，占空比越小越亮）
          __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, pulse); break;
        case K2_Pin: HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_4); break; // L7关灯
        case K3_Pin: HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_4); break; // L7亮灯
        case K4_Pin: if (pulse < 99) pulse += 2; // L7亮度减小
          __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, pulse); break;
        default: break;
      }
    }
    osDelay(100);
  }
  /* USER CODE END StartDefaultTask */
}
```

# LED灯调光应用示例——修改主菜单绘制函数

为了观察占空比变化，修改原来程序中的UIMain主菜单显示函数如右。

修改完成后，重新编译工程，下载并进行测试观察按动K1 ~ K4按键时L7的亮灯变化情况和OLED显示屏上的占空比数值是否对应。

```
void UIMain(void) {
    GUI_Clear();                    // 屏幕内容清空
    GUI_SetFont(&GUI_FontHZ_SimSun_16);     // 设置文本字体为16号宋体

    char buf[30];
    // 获取比较值和ARR值
    int cpv = __HAL_TIM_GET_COMPARE(&htim1, TIM_CHANNEL_4);
    int arr = __HAL_TIM_GET_AUTORELOAD(&htim1) + 1;

    // 屏幕正上方居中显示标题
    GUI_DispStringHCenterAt("PWM调光", 64, 0);
    // 格式化字符串
    sprintf(buf, "占空比:%.1f%%", cpv * 100.0 / arr);
    // 显示占空比
    GUI_DispStringAt(buf, 10, 30);GUI_DispStringAt(buf, 10, 30);
    GUI_Update();                   // 刷新屏幕显示
}
```

# 实验内容

**1. 示例工程实践练习**

参照本次实验讲义内容，创建并复现EX08的电子琴示例和LED调光示例工程，最后在学习板上测试通过。

**2. 定时器应用练习**

用通用定时器的定时功能也可以模拟实现PWM调光功能。设计实现一个周期为100us的定时中断，在中断中进行计数加1，计数超过100时重置为1；且判断计数小于指定亮度时8个LED灯亮，计数大于指定亮度时8个LED灯灭。

在实验内容1的基础上，重新设置TIM1定时周期，关闭PWM输出功能，使能定时溢出中断；将8个LED灯对应端口的工作模式全改回到GPIO_Output工作模式。最后重新导出MDK工程代码。根据上述设计思路，完成上述8个LED灯的调光功能，最后在学习板上测试通过。

**3.** 到学习通上的对应课程作业中，完成提交本次实验的**课堂作业**。

# 下一次实验09

## 自选实验1

请提前做好选题和预习